

CONSTANTS, VARIABLES, AND DATA TYPES

2.1 INTRODUCTION

A programming language is designed to help process certain kinds of *data* consisting of numbers, characters and strings and to provide useful output known as *information*. The task of processing of data is accomplished by executing a sequence of precise instructions called a *program*. These instructions are formed using certain symbols and words according to some rigid rules known as *syntax rules*(or *grammar*). Every program instruction must conform precisely to the syntax rules of the language.

Like any other language, C has its own vocabulary and grammar. In this chapter, we will discuss the concepts of constants and variables and their types as they relate to C programming language.

2.2 CHARACTER SET

The characters that can be used to form words, numbers and expressions depend upon the computer on which the program is run. However, a subset of characters is available that can be used on most personal, micro, mini and mainframe computers. The characters in C are grouped into the following categories:

1. Letters
2. Digits
3. Special characters
4. White spaces

The complete character set is given in Table 2.1.

The compiler ignores white spaces unless they are a part of a string constant. White spaces may be used to separate words, but are prohibited between the characters of keywords and identifiers.

Trigraph Characters

Many non-English keyboards do not support all the characters mentioned in the Table 2.1. ANSI C introduces the concept of “trigraph” sequences to provide a way to enter certain characters that are not available on some keyboards. Each trigraph sequence consists of three characters (two

Table 2.1 C Character Set

Letters	Digits
Uppercase A.....Z	All decimal digits 09
Lowercase a.....z	
Special Characters	
, comma	& ampersand
. period	^ caret
; semicolon	* asterisk
: colon	- minus sign
? question mark	+ plus sign
' apostrophe	< opening angle bracket (or less than sign)
" quotation mark	> closing angle bracket (or greater than sign)
! exclamation mark	(left paranthesis
vertical bar	) right paranthesis
/ slash	[left bracket
\ backslash	] right bracket
~ tilde	{ left brace
_ under score	} right brace
\$ dollar sign	
% per cent sign	
# number sign	

White Spaces

Blank space
Horizontal tab
Carriage return
New line
Form feed

question marks followed by another character) as shown in Table 2.2. For example, if a keyboard does not support square brackets, we can still use them in a program using the trigraphs ??(and ??).

Table 2.2 ANSI C Trigraph Sequences

Trigraph sequence	Translation
??=	# number sign
??(	[left bracket
??)	] right bracket
??<	{ left brace
??>	} right brace
??!	vertical bar
??/	\ back slash
??'	^ caret
??-	~ tilde

2.3 C TOKENS

In a passage of text, individual words and punctuation marks are called *tokens*. Similarly, in a C program the smallest individual units are known as C tokens. C has six types of tokens as shown in Fig. 2.1. C programs are written using these tokens and the syntax of the language.

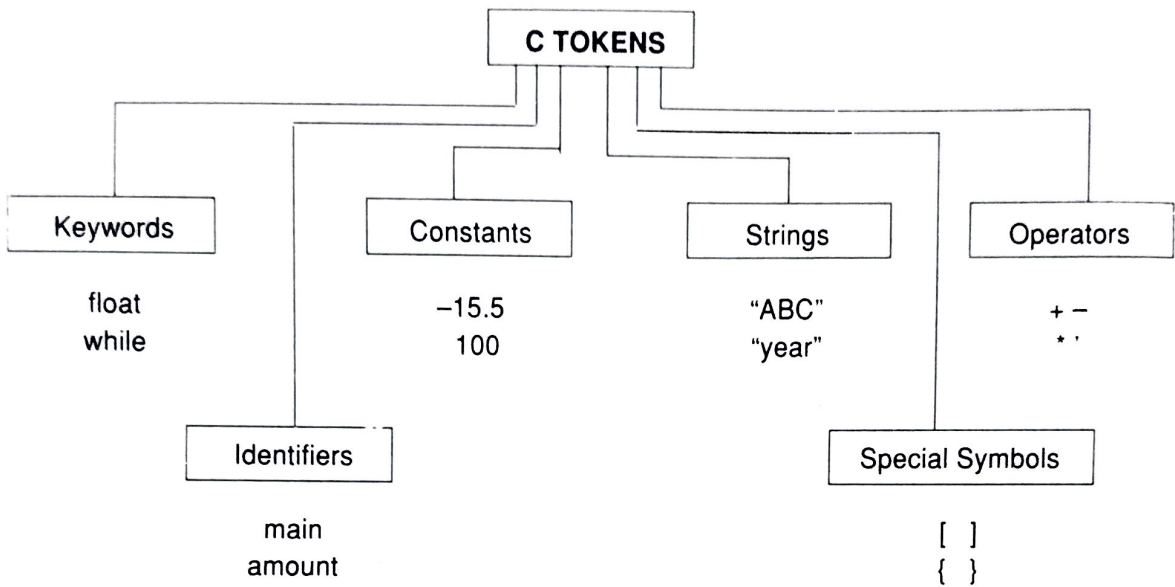


Fig. 2.1 C tokens and examples

2.4 KEYWORDS AND IDENTIFIERS

Every C word is classified as either a *keyword* or an *identifier*. All keywords have fixed meanings and these meanings cannot be changed. Keywords serve as basic building blocks for program statements. The list of all keywords in ANSI C are listed in Table 2.3. All keywords must be written in lowercase. Some compilers may use additional keywords that must be identified from the C manual.

Table 2.3 ANSI C Keywords

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Identifiers refer to the names of variables, functions and arrays. These are user-defined names and consist of a sequence of letters and digits, with a letter as a first character. Both uppercase and lowercase letters are permitted, although lowercase letters are commonly used. The underscore character is also permitted in identifiers. It is usually used as a link between two words in long identifiers.

2.5 CONSTANTS

Constants in C refer to fixed values that do not change during the execution of a program. C supports several types of constants as illustrated in Fig. 2.2.

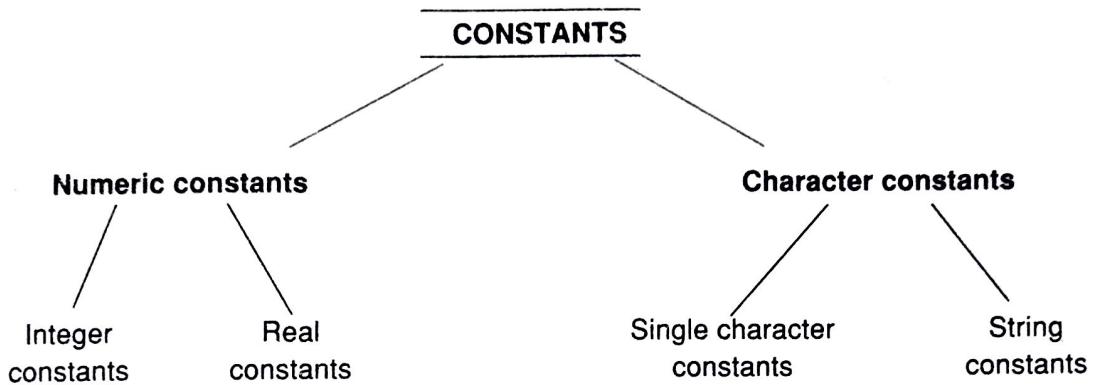


Fig. 2.2 *Basic types of C constants*

Integer Constants

An *integer* constant refers to a sequence of digits. There are three types of integers, namely, *decimal*, *octal* and *hexadecimal*.

Decimal integers consist of a set of digits, 0 through 9, preceded by an optional – or + sign. Valid examples of decimal integer constants are :

```

123
-321
0
654321
+78
  
```

Embedded spaces, commas, and non-digit characters are not permitted between digits. For example,

```

15 750
20,000
$1000
  
```

are illegal numbers. Note that ANSI C supports *unary plus* which was not defined earlier.

22 PROGRAMMING IN ANSI C

An *octal* integer constant consists of any combination of digits from the set 0 through 7, with a leading 0. Some examples of octal integers are:

037
0
0435
0551

A sequence of digits preceded by 0x or 0X is considered as *hexadecimal* integer. They may also include alphabets A through F or a through f. The letters A through F represent the numbers 10 through 15. Following are the examples of valid hex integers.

0X2
0x9F
0Xbcd
0x

We rarely use octal and hexadecimal numbers in programming.

The largest integer value that can be stored is machine-dependent. It is 32767 on 16-bit machines and 2,147,483,647 on 32-bit machines. It is also possible to store larger integer constants on these machines by appending *qualifiers* such as U, L and UL to the constants. For example:

56789U	or	56789u	(unsigned integer)
987612347UL	or	98761234ul	(unsigned long integer)
9876543L	or	9876543l	(long integer)

The concept of unsigned and long integers are discussed in detail in Section 2.7.

Example 2.1

Representation of integer constants on a 16-bit computer.

The output in Fig. 2.3 shows that the integer values larger than 32767 are not properly stored on a 16-bit machine. However, when they are qualified as long integer (by appending L), the values are correctly stored.

Program

```
/******  
/*      INTEGER NUMBERS ON 16-BIT MACHINE      */  
/******  
  
main()  
{  
    printf("Integer values\n\n");  
    printf("%d %d %d\n", 32767, 32767+1, 32767+10);  
    printf("\n");  
    printf("Long integer values\n\n");
```

```
printf("%ld %ld %ld\n", 32767L, 32767L+1L, 32767L+10L);
}
```

Output

Integer values

32767 -32768 -32759

Long integer values

32767 32768 32777

Fig. 2.3 Representation of integer constants

Real Constants

Integer numbers are inadequate to represent quantities that vary continuously, such as distances, heights, temperatures, prices, and so on. These quantities are represented by numbers containing fractional parts like 17.548. Such numbers are called *real* (or *floating point*) constants. Further examples of real constants are:

0.0083
-0.75
435.36
+247.0

These numbers are shown in *decimal notation*, having a whole number followed by a decimal point and the fractional part. It is possible to omit digits before the decimal point, or digits after the decimal point. That is,

215.
.95
-.71
+.5

are all valid real numbers.

A real number may also be expressed in *exponential* (or *scientific*) notation. For example, the value 215.65 may be written as 2.1565e2 in exponential notation. e2 means multiply by 10². The general form is:

<i>mantissa</i> • <i>exponent</i>

The *mantissa* is either a real number expressed in *decimal notation* or an integer. The *exponent* is an integer number with an optional *plus* or *minus* sign. The letter *e* separating the mantissa and the exponent can be written in either lowercase or uppercase. Since the exponent causes the decimal point to “float”, this notation is said to represent a real number in *floating point form*. Examples of legal floating point constants are:

0.65e4
 12e-2
 1.5e+5
 3.18E3
 -1.2E-1

Embedded white space is not allowed.

Exponential notation is useful for representing numbers that are either very large or very small in magnitude. For example, 7500000000 may be written as 7.5E9 or 75E8. Similarly, -0.000000368 is equivalent to -3.68E-7.

Floating point constants are normally represented as double-precision quantities. However, the suffixes f or F may be used to force single-precision and l or L to extend double-precision further.

Some examples of valid and invalid numeric constants are given in Table 2.4

Table 2.4 Examples of Numeric Constants

Constant	Valid ?	Remarks
698354L	Yes	Represents long integer
25,000	No	Comma is not allowed
+5.0E3	Yes	(ANSI C supports unary plus)
3.5e-5	Yes	
7.1e 4	No	No white space is permitted
-4.5e-2	Yes	
1.5E+2.5	No	Exponent must be an integer
\$255	No	\$ symbol is not permitted
0X7B	Yes	Hexadecimal integer

Single Character Constants

A single character constant (or simply character constant) contains a single character enclosed within a pair of *single quote marks*. Examples of character constants are:

'5' 'X' ';' ' '

Note that the character constant '5' is not the same as the *number* 5. The last constant is a blank space.

Character constants have integer values known as ASCII values. For example, the statement

```
printf("%d", 'a');
```

would print the number 97, the ASCII value of the letter a. Similarly, the statement

```
printf("%c", 97);
```

would output the letter 'a'. ASCII values for all characters are given in Appendix II.

Since each character constant represent an integer value it is also possible to perform arithmetic operations on character constants. These are discussed in Chapter 8.

String Constants

A string constant is a sequence of characters enclosed in *double* quotes. The characters may be letters, numbers, special characters and blank space. Examples are:

```
"Hello!"
"1987"
"WELL DONE"
"?...!"
"5+3"
"X"
```

Remember that a character constant (e.g., 'X') is not equivalent to the single character string constant (e.g., "X"). Further, a single character string constant does not have an equivalent integer value while a character constant has an integer value. Character strings are often used in programs to build meaningful programs. Manipulation of character strings are considered in detail in Chapter 8.

Backslash Character Constants

C supports some special backslash character constants that are used in output functions. For example, the symbol '\n' stands for newline character. A list of such backslash character constants is given in Table 2.5. Note that each one of them represents one character, although they consist of two characters. These character combinations are known as *escape sequences*.

Table 2.5 Backslash Character Constants

Constant	Meaning
'\a'	audible alert (bell)
'\b'	back space
'\f'	form feed
'\n'	new line
'\r'	carriage return
'\t'	horizontal tab
'\v'	vertical tab
'\''	single quote
'\"'	double quote
'\?'	question mark
'\\'	backslash
'\0'	null

2.6 VARIABLES

A *variable* is a data name that may be used to store a data value. Unlike constants that remain unchanged during the execution of a program, a variable may take different values at different times during execution. In Chapter 1, we used several variables. For instance, we used the variable **amount** in Sample Program 3 to store the value of money at the end of each year (after adding the interest earned during that year).

A variable name can be chosen by the programmer in a meaningful way so as to reflect its function or nature in the program. Some examples of such names are:

Average
height
Total
Counter_1
class_strength

As mentioned earlier, variable names may consist of letters, digits, and the underscore(_) character, subject to the following conditions:

- 1. They must begin with a letter. Some systems permit underscore as the first character.
- 2. ANSI standard recognizes a length of 31 characters. However, the length should not be normally more than eight characters, since only the first eight characters are treated as significant by many compilers.
- 3. Uppercase and lowercase are significant. That is, the variable **Total** is not the same as **total** or **TOTAL**.
- 4. The variable name should not be a keyword.
- 5. White space is not allowed.

Some examples of valid variable names are:

John	Value	T_raise
Delhi	x1	ph_value
mark	sum1	distance

Invalid examples include;

123	(area)
% .	25th

Further examples of variable names and their correctness are given in Table 2.6.

Table 2.6 Examples of Variable Names

Variable name	Valid ?	Remark
First_tag	Valid	
char	Not valid	char is a keyword
Price\$	Not valid	Dollar sign is illegal
group one	Not valid	Blank space is not permitted
average_number	Valid	First eight characters are significant
int_type	Valid	Keyword may be part of a name

If only the first eight characters are recognized by a compiler, then the two names

average_height
average_weight

mean the same thing to the computer. Such names can be rewritten as

avg_height and **avg_weight**

or

ht_average and **wt_average**

without changing their meanings.

2.7 DATA TYPES

C language is rich in its *data types*. Storage representations and machine instructions to handle constants differ from machine to machine. The variety of data types available allow the programmer to select the type appropriate to the needs of the application as well as the machine.

ANSI C supports four classes of data types :

1. Primary (or fundamental) data types
2. User-defined data types
3. Derived data types
4. Empty data set

The primary data types and their extensions are discussed in this section. The user-defined data types are defined in the next section while the derived data types such as arrays, functions, structures and pointers are discussed as and when they are encountered. The empty data set is discussed in the chapter on functions.

All C compilers support four fundamental data types, namely integer (**int**), character (**char**), floating point (**float**), and double-precision floating point (**double**). Many of them also offer extended data types such as **long int** and **long double**. Various data types and the terminology used to describe them are given in Fig. 2.4. The range of the basic four types are given in Table 2.7. We discuss briefly each one of them in this section.

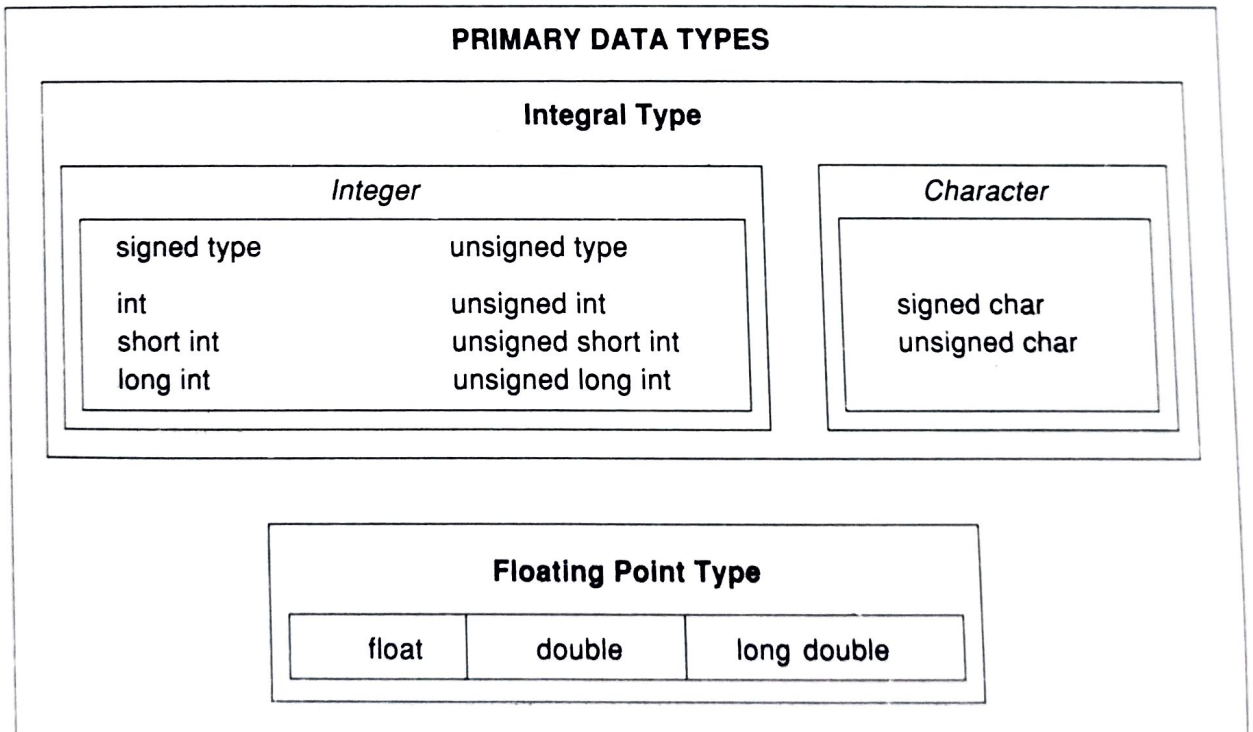


Fig. 2.4 Primary data types in C

Table 2.7 Size and Range of Basic Data Types

Data type	Range of values
char	-128 to 127
int	-32,768 to 32,767
float	3.4e-38 to 3.4e+38
double	1.7e-308 to 1.7e+308

Integer Types

Integers are whole numbers with a range of values supported by a particular machine. Generally, integers occupy one word of storage, and since the word sizes of machines vary (typically, 16 or 32 bits) the size of an integer that can be stored depends on the computer. If we use a 16 bit word length, the size of the integer value is limited to the range -32768 to +32767 (that is, -2^{15} to $+2^{15}-1$). A signed integer uses one bit for sign and 15 bits for the magnitude of the number. Similarly, a 32 bit word length can store an integer ranging from -2,147,483,648 to 2,147,483,647.

In order to provide some control over the range of numbers and storage space, C has three classes of integer storage, namely **short int**, **int**, and **long int**, in both **signed** and **unsigned** forms. For example, **short int** represents fairly small integer values and requires half the amount of storage as a regular **int** number uses. Unlike signed integers, unsigned integers use all the bits for the magnitude of the number and are always positive. Therefore, for a 16 bit machine, the range of unsigned integer numbers will be from 0 to 65,535.

We declare **long** and **unsigned** integers to increase the range of values. The use of qualifier **signed** on integers is optional because the default declaration assumes a signed number. Table 2.8 shows all the allowed combinations of basic types and qualifiers and their size and range on a 16-bit machine.

Table 2.8 Size and Range of Data Types on a 16-bit Machine

Type	Size (bits)	Range
char or signed char	8	-128 to 127
unsigned char	8	0 to 255
int or signed int	16	-32,768 to 32,767
unsigned int	16	0 to 65535
short int or signed short int	8	-128 to 127
unsigned short int	8	0 to 255
long int or signed long int	32	-2,147,483,648 to 2,147,483,647
unsigned long int	32	0 to 4,294,967,295
float	32	3.4E-38 to 3.4E+38
double	64	1.7E-308 to 1.7E+308
long double	80	3.4E-4932 to 1.1E+4932

Floating Point Types

Floating point (or real) numbers are stored in 32 bits (on all 16 bit and 32 bit machines), with 6 digits of precision. Floating point numbers are defined in C by the keyword **float**. When the accuracy provided by a **float** number is not sufficient, the type **double** can be used to define the number. A **double** data type number uses 64 bits giving a precision of 14 digits. These are known as *double precision* numbers. Remember that **double** type represents the same data type that **float** represents, but with a greater precision. To extend the precision further, we may use **long double** which uses 80 bits.

Character Types

A single character can be defined as a character(**char**) type data. Characters are usually stored in 8 bits (one byte) of internal storage. The qualifier **signed** or **unsigned** may be explicitly applied to **char**. While **unsigned chars** have values between 0 and 255, **signed char**s have values from -128 to 127.

2.8 DECLARATION OF VARIABLES

After designing suitable variable names, we must declare them to the compiler. Declaration does two things:

1. It tells the compiler what the variable name is.
2. It specifies what type of data the variable will hold.

The declaration of variables must be done before they are used in the program.

Primary Type Declaration

A variable can be used to store a value of any data type. That is, the name has nothing to do with its type. The syntax for declaring a variable is as follows:

data-type v1,v2,...vn ;

v1, v2, ..., vn are the names of variables. Variables are separated by commas. A declaration statement must end with a semicolon. For example, valid declarations are:

```
int count;
int number, total;
double ratio;
```

int and **double** are the keywords to represent integer type and real type data values respectively. Table 2.9 shows various data types and their keyword equivalents.

Table 2.9 Data Types and Their Keywords

Data type	Keyword equivalent
Character	char
Unsigned character	unsigned char

generally, 6 or 32 length (5-1). A 32 bit classes is. For range as for the range of talifier ble 2.8 range



Signed character	signed char
Signed integer	signed int (or int)
Signed short integer	signed short int (or short int or short)
Signed long integer	signed long int (or long int or long)
Unsigned integer	unsigned int (or unsigned)
Unsigned short integer	unsigned short int (or unsigned short)
Unsigned long integer	unsigned long int (or unsigned long)
Floating point	float
Double-precision floating point	double
Extended double-precision floating point	long double

The program segment given in Fig. 2.5 illustrates declaration of variables. **main()** is the beginning of the program. The opening brace { signals the execution of the program. Declaration of variables is usually done immediately after the opening brace of the program. The variables can also be declared outside (either before or after) the **main** function. The importance of place of declaration will be dealt in detail later while discussing functions.

```

main() /*.....Program Name.....*/
{
/*.....Declaration.....*/

    float          x, y;
    int            code;
    short int      count;
    long int       amount;
    double         deviation;
    unsigned       n;
    char           c;
/*.....Computation.....*/
    .
    ....
    ....
    ....
} /*.....Program ends.....*/

```

Fig. 2.5 Declaration of variables

When an adjective (qualifier) **short**, **long**, or **unsigned** is used without a basic data type specifier, C compilers treat the data type as an **int**. If we want to declare a character variable as unsigned, then we must do so using both the terms like **unsigned char**.

User-Defined Type Declaration

C supports a feature known as “type definition” that allows users to define an identifier that would represent an existing data type. The user-defined data type identifier can later be used to declare variables. It takes the general form:

```
typedef type identifier;
```

Where *type* refers to an existing data type and “identifier” refers to the “new” name given to the data type. The existing data type may belong to any class of type, including the user-defined ones. Remember that the new type is ‘new’ only in name, but not the data type. **typedef** cannot create a new type. Some examples of type definition are:

```
typedef int units;  
typedef float marks;
```

Here, **units** symbolizes **int** and **marks** symbolizes **float**. They can be later used to declare variables as follows:

```
units batch1, batch2;  
marks name1[50], name2[50];
```

batch1 and **batch2** are declared as **int** variable and **name1**[50] and **name2**[50] are declared as 50 element floating point array variables. The main advantage of **typedef** is that we can create meaningful data type names for increasing the readability of the program.

Another user-defined data type is enumerated data type provided by ANSI standard. It is defined as follows:

```
enum identifier {value1, value2, ... valuen};
```

The “identifier” is a user-defined enumerated data type which can be used to declare variables that can have one of the values enclosed within the braces (known as *enumeration constants*). After this definition, we can declare variables to be of this ‘new’ type as below:

```
enum identifier v1, v2, ... vn;
```

The enumerated variables *v1*, *v2*, ... *vn* can only have one of the values *value1*, *value2*, ... *valuen*. The assignments of the following types are valid:

```
v1 = value3;  
v5 = value1;
```

An example:

```
enum day {Monday, Tuesday, ... Sunday};  
enum day week_st, week_end;
```

```
week_st = Monday;  
week_end = Friday;  
if(week_st == Tuesday)  
    week_end = Saturday;
```

32 PROGRAMMING IN ANSI C

The compiler automatically assigns integer digits beginning with 0 to all the enumeration constants. That is, the enumeration constant *value1* is assigned 0, *value2* is assigned 1, and so on. However, the automatic assignments can be overridden by assigning values explicitly to the enumeration constants. For example:

```
enum day {Monday = 1, Tuesday, ... Sunday};
```

Here, the constant Monday is assigned the value of 1. The remaining constants are assigned values that increase successively by 1.

The definition and declaration of enumerated variables can be combined in one statement. Example:

```
enum day {Monday, ... Sunday} week_st, week_end;
```

Declaration of Storage Class

Variables in C can have not only *data type* but also *storage class* that provides information about their location and visibility. The storage class decides the portion of the program within which the variables are recognized. Consider the following example:

```
/* Example of storage classes */  
  
int m;  
  
main()  
{  
    int i;  
    float balance;  
    ....  
    ....  
    function1();  
}  
  
function1()  
{  
    int i;  
    float sum;  
    ....  
    ....  
}
```

The variable **m** which has been declared before the **main** is called *global* variable. It can be used in all the functions in the program. It need not be declared in other functions. A global variable is also known as an *external* variable.

The variables **i**, **balance** and **sum** are called *local* variables because they are declared inside a function. Local variables are visible and meaningful only inside the functions in which they are declared. They are not known to other functions. Note that the variable **i** has been declared in both the functions. Any change in the value of **i** in one function does not affect its value in the other.

C provides a variety of storage class specifiers that can be used to declare explicitly the scope and lifetime of variables. The concepts of scope and lifetime are important only in multifunction

and multiple file programs and therefore the storage classes are considered in detail later when functions are discussed. For now, remember that there are four storage class specifiers (**auto**, **register**, **static**, and **extern**) whose meanings are given in Table 2.10.

Table 2.10 Storage Classes and Their Meaning

Storage class	Meaning
auto	Local variable known to only to the function in which it is declared. <i>Default is auto.</i>
static	Local variable which exists and retains its value even after the control is transferred to the calling function.
extern	Global variable known to all functions in the file.
register	Local variable which is stored in the register.

The storage class is another qualifier (like **long** or **unsigned**) that can be added to a variable declaration as shown below:

```

auto int count;
register char ch;
static int x;
extern long total;

```

Static and external (**extern**) variables are automatically initialized to zero. Automatic (**auto**) variables contain undefined values (known as 'garbage') unless they are initialized explicitly.

2.9 ASSIGNING VALUES TO VARIABLES

Variables are created for use in program statements such as

```

value = amount + inrate * amount;
while (year <= PERIOD)
{
    ....
    ....
    year = year + 1;
}

```

In the first statement, the numeric value stored in the variable **inrate** is multiplied by the value stored in **amount** and the product is added to **amount**. The result is stored in the variable **value**. This process is possible only if the variables **amount** and **inrate** have already been given values. The variable **value** is called the *target variable*. While all the variables are declared for their type, the variables that are used in expressions (on the right side of equal (=) sign of a computational statement) *must* be assigned values before they are encountered in the program. Similarly, the variable **year** and the symbolic constant **PERIOD** in the **while** statement must be assigned values before this statement is encountered.

Assignment Statement

Values can be assigned to variables using the assignment operator = as follows:

`variable_name = constant;`

We have already used such statements in Chapter 1. Further examples are:

```
initial_value    =    0;
final_value     =    100;
balance         =    75.84;
yes             =    'x';
```

C permits multiple assignments in one line. For example

```
initial_value = 0; final_value = 100;
```

are valid statements.

An assignment statement implies that the value of the variable on the left of the 'equal sign' is set equal to the value of the quantity (or the expression) on the right. The statement

```
year = year + 1;
```

means that the 'new value' of **year** is equal to the 'old value' of **year** plus 1.

During assignment operation, C converts the type of value on the right-hand side to the type on the left. This may involve truncation when real value is converted to an integer.

It is also possible to assign a value to a variable at the time the variable is declared. This takes the following form:

`data-type variable_name = constant;`

Some examples are:

```
int final_value = 100;
char yes       = 'x';
double balance = 75.84;
```

The process of giving initial values to variables is called *initialization*. C permits the initialization of more than one variables in one statement using multiple assignment operators. For example the statements

```
p = q = s = 0;
x = y = z = MAX;
```

are valid. The first statement initializes the variables **p**, **q**, and **s** to zero while the second initializes **x**, **y**, and **z** with **MAX**. Note that **MAX** is a symbolic constant defined at the beginning.

Remember that external and static variables are initialized to zero by *default*. Automatic variables that are not initialized explicitly will contain garbage.

Example 2.2

Program in Fig. 2.6 shows typical declarations, assignments and values stored in various types of variables.

The variables **x** and **p** have been declared as floating-point variables. Note that the way the value

Program

```

...../
/*      EXAMPLES OF ASSIGNMENTS      */
...../

main()
{
/*.....DECLARATIONS.....*/

    float    x, p ;
    double   y, q ;
    unsigned k ;

/*.....DECLARATIONS AND ASSIGNMENTS.....*/

    int      m = 54321 ;
    long int  n = 1234567890 ;

/*.....ASSIGNMENTS.....*/

    x = 1.234567890000 ;
    y = 9.87654321 ;
    k = 54321 ;
    p = q = 1.0 ;

/*.....PRINTING.....*/

    printf("m = %d\n", m) ;
    printf("n = %ld\n", n) ;
    printf("x = %.12lf\n", x) ;
    printf("x = %f\n", x) ;
    printf("y = %.12lf\n", y) ;
    printf("y = %f\n", y) ;
    printf("k = %u  p = %f  q = %.12lf\n", k, p, q) ;
}

```

Output

```

m = -11215
n = 1234567890
x = 1.234567880630
x = 1.234568
y = 9.876543210000
y = 9.876543
k = 54321  p = 1.000000  q = 1.000000000000

```

Fig. 2.6 *Examples of assignments*

of 1.234567890000 that we assigned to **x** is displayed under different output formats. The value of **x** is displayed as 1.234567880630 under `%12lf` format, while the actual value assigned is 1.234567890000. This is because the variable **x** has been declared as a **float** that can store values only upto six decimal places.

The variable **m** that has been declared as **int** is not able to store the value 54321 correctly. Instead, it contains some garbage. Since this program was run on a 16-bit machine, the maximum value that an **int** variable can store is only 32767. However, the variable **k** (declared as **unsigned**) has stored the value 54321 correctly. Similarly, the **long int** variable **n** has stored the value 1234567890 correctly.

The value 9.87654321 assigned to **y** declared as **double** has been stored correctly but the value is printed as 9.876543 under `%lf` format. Note that unless specified otherwise, the **printf** function will always display a **float** or **double** value to six decimal places. We will discuss later the output formats for displaying numbers.

Declaring a Variable as Constant

We may like the value of certain variables to remain constant during the execution of a program. We can achieve this by declaring the variable with the qualifier **const** at the time of initialization. Example:

```
const int class_size = 40;
```

const is a new data type qualifier defined by ANSI standard. This tells the compiler that the value of the **int** variable **class_size** must not be modified by the program. However, it can be used on the right-hand side of an assignment statement like any other variable.

Declaring a Variable as Volatile

ANSI standard defines another qualifier **volatile** that could be used to tell explicitly the compiler that a variable's value may be changed at any time by some external sources (from outside the program). For example:

```
volatile int date;
```

The value of **date** may be altered by some external factors even if it does not appear on the left-hand side of an assignment statement. When we declare a variable as **volatile**, the compiler will examine the value of the variable each time it is encountered to see whether any external alteration has changed the value.

Remember that the value of a variable declared as **volatile** can be modified by its own program as well. If we wish that the value must not be modified by the program while it may be altered by some other process, then we may declare the variable as both **const** and **volatile** as shown below:

```
volatile const int location = 100;
```

Overflow and Underflow of Data

Problem of data overflow occurs when the value of a variable is either too big or too small for the data type to hold. The largest value that a variable can hold also depends on the machine. Since

floating point values are rounded off to the number of significant digits allowed (or specified), an overflow normally results in the largest possible real value, whereas an underflow results in zero.

Integers are always exact within the limits of the range of the integral data types used. However, an overflow which is a serious problem may occur if the data type does not match the value of the constant. C does not provide any warning or indication of integer overflow. It simply gives incorrect results. (Overflow normally produces a negative number.) We should therefore exercise a greater care to define correct data types for handling the input/output values.

Reading Data from Keyboard

Another way of giving values to variables is to input data through keyboard using the **scanf** function. It is a general input function available in C and is very similar in concept to the **printf** function. It works much like an INPUT statement in BASIC. The general format of **scanf** is as follows:

```
scanf("control string", &variable1, &variable2,...);
```

The control string contains the format of data being received. The ampersand symbol **&** before each variable name is an operator that specifies the variable name's *address*. We must always use this operator, otherwise unexpected results may occur. Let us look at an example:

```
scanf("%d", &number);
```

When this statement is encountered by the computer, the execution stops and waits for the value of the variable **number** to be typed in. Since the control string "**%d**" specifies that an integer value is to be read from the terminal, we have to type in the value in integer form. Once the number is typed in and the 'Return' Key is pressed, the computer then proceeds to the next statement. Thus, the use of **scanf** provides an interactive feature and makes the program 'user friendly'.

Example 2.3

The program in Fig. 2.7 illustrates the use of **scanf** function.

The first executable statement in the program is a **printf**, requesting the user to enter an integer number. This is known as "prompt message" and appears on the screen like

```
Enter an integer number
```

As soon as the user types in an integer number, the computer proceeds to compare the value with 100. If the value typed in is less than 100, then a message

```
Your number is smaller than 100
```

is printed on the screen. Otherwise, the message

```
Your number contains more than two digits
```

is printed. Outputs of the program, run for two different inputs are also shown in Fig. 2.7.



Program

```

...../
/*  INTERACTIVE COMPUTING USING scanf FUNCTION  */
...../

main()
{
    int number;

    printf("Enter an integer number\n");
    scanf ("%d", &number);

    if ( number < 100 )
        printf("Your number is smaller than 100\n\n");
    else
        printf("Your number contains more than two digits\n");
}

```

Output

```

Enter an integer number
54
Your number is smaller than 100

Enter an integer number
108
Your number contains more than two digits

```

Fig. 2.7 Use of *scanf* function

Some compilers permit the use of the 'prompt message' as a part of the control string in *scanf*, like

```
scanf("Enter a number %d",&number);
```

We discuss more about *scanf* in Chapter 4.

In Fig. 2.7 we have used a decision statement *if...else* to decide whether the number is less than 100. Decision statements are discussed in depth in Chapter 5.

Example 2.4

Sample Program 3 discussed in Chapter 1 can be converted into a more flexible interactive program using *scanf* as shown in Fig. 2.8.

In this case, computer requests the user to input the values of the amount to be invested, interest rate

rate and period of investment by printing a prompt message

Input amount, interest rate, and period

and then waits for input values. As soon as we finish entering the three values

Program

```

/...../
/*    INTERACTIVE INVESTMENT PROGRAM    */
/...../

main()
{
    int  year, period ;
    float amount, inrate, value ;

    printf("Input amount, interest rate, and period\n\n") ;
    scanf ("%f %f %d", &amount, &inrate, &period) ;
    printf("\n") ;
    year = 1 ;

    while( year <= period )
    {
        value = amount + inrate * amount ;
        printf("%2d Rs %8.2f\n", year, value) ;
        amount = value ;
        year = year + 1 ;
    }
}

```

Output

Input amount, interest rate, and period

10000 0.14 5

```

1  Rs 11400.00
2  Rs 12996.00
3  Rs 14815.44
4  Rs 16889.60
5  Rs 19254.15

```

Input amount, interest rate, and period

```

20000 0.12 7

1   Rs 22400.00
2   Rs 25088.00
3   Rs 28098.56
4   Rs 31470.39
5   Rs 35246.84
6   Rs 39476.46
7   Rs 44213.63

```

Fig. 2.8 *Interactive investment program*

corresponding to the three variables **amount**, **inrate**, and **period**, the computer begins to calculate the amount at the end of each year, upto 'period' and produces output as shown in Fig. 2.8.

Note that the **scanf** function contains three variables. In such cases, care should be exercised to see that the values entered match the *order* and *type* of the variables in the list. Any mismatch might lead to unexpected results. The compiler may not detect such errors.

2.10 DEFINING SYMBOLIC CONSTANTS

We often use certain unique constants in a program. These constants may appear repeatedly in a number of places in the program. One example of such a constant is 3.142, representing the value of the mathematical constant "pi". Another example is the total number of students whose mark-sheets are analysed by a 'test analysis program'. The number of students, say 50, may be used for calculating the class total, class average, standard deviation, etc. We face two problems in the subsequent use of such programs.

1. Problem in modification of the program.
2. Problem in understanding the program.

Modifiability

We may like to change the value of "pi" from 3.142 to 3.14159 to improve the accuracy of calculations or the number 50 to 100 to process the test results of another class. In both the cases, we will have to search throughout the program and explicitly change the value of the constant wherever it has been used. If any value is left unchanged, the program may produce disastrous outputs.

Understandability

When a numeric value appears in a program, its use is not always clear, especially when the same value means different things in different places. For example, the number 50 may mean the number of students at one place and the 'pass marks' at another place of the same program. We may forget what a certain number meant, when we read the program some days later.

Assignment of such constants to a *symbolic name* frees us from these problems. For example, we may use the name **STRENGTH** to define the number of students and **PASS_MARK** to define the pass marks required in a subject. Constant values are assigned to these names at the beginning

of the program. Subsequent use of the names **STRENGTH** and **PASS_MARK** in the program has the effect of causing their defined values to be automatically substituted at the appropriate points. A constant is defined as follows:

#define *symbolic-name* *value of constant*

Valid examples of constant definitions are:

```
#define STRENGTH      100
#define PASS_MARK     50
#define MAX           200
#define PI             3.14159
```

Symbolic names are sometimes called *constant identifiers*. Since the symbolic names are constants (not variables), they do not appear in declarations. The following rules apply to a **#define** statement which define a symbolic constant.

1. Symbolic names have the same form as variable names. (Symbolic names are written in CAPITALS to visually distinguish them from the normal variable names which are written in lowercase letters. This is only a convention, not a rule.)
2. No blank space between the pound sign '#' and the word **define** is permitted.
3. '#' must be the first character in the line.
4. A blank space is required between **#define** and *symbolic name* and between the *symbolic name* and the *constant*.
5. **#define** statements must not end with a semicolon.
6. After definition, the *symbolic name* should not be assigned any other value within the program by using an assignment statement. For example, **STRENGTH = 200;** is illegal.
7. Symbolic names are NOT declared for data types. Its data type depends on the type of constant.
8. **#define** statements may appear *anywhere* in the program but before it is referenced in the program (the usual practice is to place them in the beginning of the program).

#define statement is a *preprocessor* compiler directive and is much more powerful than what has been mentioned here. More advanced types of definitions will be discussed later. Table 2.11 illustrates some invalid statements of **#define**.

Table 2.11 Examples of Invalid #define Statements

Statement	Validity	Remark
#define X = 2.5	Invalid	'=' sign is not allowed
# define MAX 10	Invalid	No white space between # and define
#define N 25;	Invalid	No semicolon at the end
#define N 5, M 10	Invalid	A statement can define only one name.
#Define ARRAY 11	Invalid	define should be in lowercase letters
#define PRICE\$ 100	Invalid	\$ symbol is not permitted in name

CASE STUDIES

1. Calculation of Average of Numbers

A program to calculate the average of a set of N numbers is given in Fig. 2.9.

Program

```

/*****
/*      AVERAGE OF n VALUES      */
*****/

#define    N 10                /* SYMBOLIC CONSTANT */

main()
{
    int  count ;                /* DECLARATION OF */
    float sum, average, number ; /* VARIABLES */

    sum = 0 ;                  /* INITIALIZATION */
    count = 0                  /* OF VARIABLES */

    while( count < N )
    {
        scanf("%f", &number) ;
        sum = sum + number ;
        count = count + 1 ;
    }

    average = sum/N ;
    printf("N = %d Sum = %f", N, sum);
    printf(" Average = %f", average);
}

```

Output

```

1
2.3
4.67
1.42
7
3.67
4.08
2.2
4.25
8.21
N = 10 Sum = 38.799999 Average = 3.880000

```

Fig. 2.9 Average of N numbers

The variable **number** is declared as **float** and therefore it can take both integer and real numbers. Since the symbolic constant **N** is assigned the value of 10 using the **#define** statement, the program accepts ten values and calculates their sum using the **while** loop. The variable **count** counts the number of values and as soon as it becomes 11, the **while** loop is exited and then the average is calculated.

Notice that the actual value of sum is 38.8 but the value displayed is 38.799999. In fact, the actual value that is displayed is quite dependent on the computer system. Such an inaccuracy is due to the way the floating point numbers are internally represented inside the computer.

2. Temperature Conversion Problem

The program presented in Fig. 2.10 converts the given temperature in fahrenheit to celsius using the following conversion formula:

$$C = \frac{F - 32}{1.8}$$

Program

```

/******
/*      FAHRENHEIT - CELSIUS CONVERSION TABLE      */
/******

#define F_LOW    0          /* ***** */
#define F_MAX    250        /* SYMBOLIC CONSTANTS */
#define STEP     25         /* ***** */

main()
{
    typedef float REAL ;          /* TYPE DEFINITION */
    REAL  fahrenheit, celsius ;    /* DECLARATION */

    fahrenheit = F_LOW ;          /* INITIALIZATION */
    printf("Fahrenheit      Celsius\n\n") ;
    while( fahrenheit <= F_MAX )
    {
        celsius = ( fahrenheit - 32.0 ) / 1.8 ;
        printf(" %5.1f    %7.2f\n", fahrenheit, celsius);
        fahrenheit = fahrenheit + STEP ;
    }
}

```

<i>Output</i>	
Fahrenheit	Celsius
0.0	-17.78
25.0	-3.89
50.0	10.00
75.0	23.89
100.0	37.78
125.0	51.67
150.0	65.56
175.0	79.44
200.0	93.33
225.0	107.22
250.0	121.11

Fig. 2.10 *Temperature conversion*

The program prints a conversion table for reading temperature in celsius, given the fahrenheit values. The minimum and maximum values and step size are defined as symbolic constants. These values can be changed by redefining the **#define** statements. A user-defined data type name **REAL** is used to declare the variables **fahrenheit** and **celsius**.

The formation specifications **%5.1f** and **%7.2** in the second **printf** statement produces two column output as shown.

REVIEW QUESTIONS AND EXERCISES

- 2.1 What are trigraph characters? How are they useful?
- 2.2 Describe the four basic data types. How could we extend the range of values they represent?
- 2.3 What is an unsigned integer constant? What is the significance of declaring a constant unsigned?
- 2.4 Describe the characteristics and purpose of escape sequence characters.
- 2.5 What is a variable and what is meant by the "value" of a variable?
- 2.6 How do variables and symbolic names differ?
- 2.7 State the differences between the declaration of a variable and the definition of a symbolic name.
- 2.8 What is initialization? Why is it important?
- 2.9 What are the qualifiers that an int can have at a time?
- 2.10 A programmer would like to use the word DPR to declare all the double-precision floating point values in his program. How could he achieve this?

- 2.11 What are enumeration variables? How are they declared? What is the advantage of using them in a program?
- 2.12 Describe the purpose of the qualifiers **const** and **volatile**.
- 2.13 When dealing with very small or very large numbers, what steps would you take to improve the accuracy of the calculations?
- 2.14 Which of the following are invalid constants and why?

0.0001	5x1.5	99999
+100	75.45 E-2	"15.75"
-45.6	-1.79 e + 4	0.00001234

- 2.15 Which of the following are invalid variable names and why?

Minimum	First.name	n1+n2	&name
doubles	3rd_row	n\$	Row1
float	Sum Total	Row Total	Column-total

- 2.16 Find errors, if any, in the following declaration statements.

```

int x;
float letter,DIGIT;
double = p,q
exponent alpha, beta;
m, n, z: INTEGER
short char c;
long int m; count;
long float temp;
    
```

- 2.17 What would be the value of x after execution of the following statements?

```

int x, y = 10;
char z = 'a';

x = y + z;
    
```

- 2.18 Identify syntax errors in the following program. After corrections, what output would you expect when you execute it?

```

#define PI 3.14159
main()
{
    int R,C;                /* R-Radius of circle */
    float perimeter;        /* Circumference of circle */
    float area;            /* area of circle */
    C = PI
    R = 5;
    Perimeter = 2.0 * C *R;
    Area      = C*R*R;
    printf("%f", "%d",&perimeter,&area)}
    
```

46 PROGRAMMING IN ANSI C

- 2.19 Write a program to determine and print the sum of the following harmonic series for a given value of n:

$$1 + 1/2 + 1/3 + \dots + 1/n$$

The value of n should be given interactively through the terminal.

- 2.20 Write a program to read the price of an item in decimal form (like 15.95) and print the output in paise (like 1595 paise).